PART 2

Indexing and evaluation

Git repository and Tag

Repository: https://github.com/froggyalchemist/irwa-search-engine
Tag: IRWA-2026-part-2

AI Use Declaration

We used **Claude Sonnet 5.5** to improve the writing style in this report. We wrote all main ideas ourselves and asked Claude to make our text more structured and concise.

We used **ChatGPT (Sol)** for certain specific complex coding review with our ideas of implementation and text polishing. The precision-recall curve code was asked to this model too.

Git repository and Tag **1**
AI Use Declaration **1**
Part 1: Indexing **2**
1. Index creation 2
2. Test queries 2
3. TF-IDF algorithm and document ranking 3
TF-IDF 3
Ranking 4
Part 2: Evaluation **5**
1. Implementation of evaluation metrics 5
2. Applying evaluation metrics 6
3. Ground truth for our test queries 7
a. Labelling results manually 7
b. Evaluation metrics for our test queries 7
c. Analysis of the current system 10

Part 1: Indexing

1. Index creation

We create the index using the product information we pre-processed in part 1. Load csv file and create index using function create_index(). Initially, no tf-idf information, we focus on counting term appearances correctly.

Our index is a nested dictionary. On the first level, the keys are the terms (e.g. cotton), and on the second the keys are the fields where this term shows up (e.g. title). The value for the fields is a list of postings [pid, position] specifying in which product and position the term appears.

```
Python
index["cotton"] = {
   "title": [
      [pid1, positions],
      [pid2, positions],
   ],
   "description": [
      [doc1, positions],
   ],
   ...
}
```

This allows us to preserve the distinction between fields in the database we were given, which will allow us to assign different weights to different fields in the ranking stage.

2. Test queries

Next, we test the searching function in the index with the five test queries we present in Table 1 below. We chose them based on EDA in part 1, mixing popular and more unpopular terms. We used popular words in order to get results (i.e. there would be no point in searching “prada luxury handbag” since there are no prada products). At the same time we want to include some rarer words so that we don’t get too many results. we don’t want super generic queries, like “men’s shirt” because then everything will be relevant.

As stated in the supplied project instructions, we use conjunctive queries: the output is the list of documents that contain ALL of the words from a query. Every query goes through the same preprocessing function as the terms, i.e. lowercasing, stemming, etc. Then, for each term in the query we get a list of products that match, and then take the intersection of all lists of product ids. In the following table we can see the results of matching products (pids) of our queries:

Query ID	Query	# of matching products	Match rate
1	men's cotton checkered shirt	401	1.43%
2	multicolor women casual t-shirt	70	0.25%
3	black sports shoes	94	0.33%
4	soft jacket for women	52	0.19%
5	grey hoodie with pockets	28	0.10%

Table 1. Our five test queries. For each query, the number of products that contain all query terms and match rate (matching products / total number of products).

3. TF-IDF algorithm and document ranking

TF-IDF

We reused the TF-IDF functions that appear in the practical sessions notebooks, but adapted them to preserve separate product fields. As we mentioned in the previous lab, we kept the fields separate to have the possibility of giving different weights to each. For now, we will give equal weight to all of them, but we could tune them in the future if needed. Here we explain the implementation with this field distinction.

The total number of occurrences of a term in a document corresponds to the occurrences in every field:

$$c_{t,d} = \sum_{f} c_{t,f,d}$$

We normalize using the product's total term-frequency norm:

$$tf_{t,f,d} = \frac{c_{t,f,d}}{\sqrt{\sum_{u \in V} c_{u,d}^2}}$$

where V is the collection vocabulary. We can see that the denominator is the same for all term-field pairs that belong to the same product.

The document frequency (df_t) counts the number of distinct products containing a term. This means that a term contributes only once to this count even when the term occurs in several fields. As usual, we define inverse document frequency as:

$$idf_t = \log\left(\frac{N}{df_t}\right)$$

Ranking

With ranking we also follow the TF-ID procedure present in the practical sessions' notebooks. We just adapt it to allow us to combine contributions from separate product fields:

$$v_{d,t} = idf_t \sum_f w_f \, tf_{t,f,d}$$

where w_f corresponds to the weight of field f. As we said before, for now we use equal weights, but our implementation allows us to change this configuration in the future.

As seen in class, we commonly use cosine similarity to compare query and document vectors. As defined in the 2nd practice session notebook, we compute the score of each query-document pair as a dot product:

$$\text{score}(q, d) = \sum_{t \in q} v_{q,t} v_{d,t}$$

And with this, we return products containing all processed query terms in descending score order. In figure 1 below we show an example of such a result for the query "black sports shoes":



Figure 1. Test query 3 search results. Product title, brand, category , pid shown and description in grey below.

Part 2: Evaluation

1. Implementation of evaluation metrics

We adapted the evaluation functions from the 2nd practical sessions and evaluated the results at K = 0. For relevance, we take a binary approach: 1 indicates the returned product satisfies the information need of the query, and 0 indicates the opposite.

Let rel_i be the relevance label of the product at position i of the returned documents, and let R be the total number of relevant products in the set. We have the number of relevant products within the first K positions as:

$$r_K = \sum_{i=1}^{K} rel_i$$

We use the following metrics regarding this:

Precision and recall	$P@K = \frac{r_K}{K}$ $R@K = \frac{r_K}{R}$

F1, combining the harmonic mean of precision and recall	$$F1@K = \frac{2(P@K)(R@K)}{P@K + R@K}$$
AP, averaging precision values at relevant positions. We also compute the mean: MAP.	$$AP@K = \frac{1}{r_K} \sum_{i=1}^{K} (P@i)\, rel_i$$
Reciprocal rank, defining the reciprocal of the position of the first relevant product. We also compute the mean: MRR.	$$RR@K = \begin{cases} \frac{1}{i^*}, & \text{if the first relevant product occurs at } i^* \leq K, \\ 0, & \text{otherwise.} \end{cases}$$
Cumulative gain metrics, comparing the obtained ranking with the ideal one	$$DCG@K = \sum_{i=1}^{K} \frac{2^{rel_i} - 1}{\log_2(i+1)}$$ $$NDCG@K = \frac{DCG@K}{IDCG@K}$$

2. Applying evaluation metrics

We apply the functions we defined earlier and compute metrics for k=10. We assume the items in validation_labels.csv are already ranked for each query: i.e row 0 is the most relevant product for query 1, row 1 is the second most relevant product, and so on. Accordingly, we assign a score to each product inversely proportional to position.

query_id	query text		R@10	AP@10	F1@10	RR@10	NDCG@10
1	women full sleeve sweatshirt cotton	0.700	0.538	0.725	0.609	1.000	0.703
2	men slim jeans blue	0.600	0.600	0.680	0.600	1.000	0.615

Table 2. per-query evaluation metrics. Each query has 20 items.

MAP@10	MRR@10
0.703	1.000

Table 3. aggregate evaluation metrics.

3. Ground truth for our test queries

a. Labelling results manually

Now we label by hand the results of our 5 test queries and evaluate the system again with those labels.

We export a CSV with the top 20 results of each test query, ranked with TF-IDF. Each row of the CSV corresponds to a (*query*, *product*) pair, and has the following columns:
- title of the product
- description of the product. We include it because it's inaccurate to judge the relevance of a product based only on its title.
- pid of the product
- query_id of the query that was used to search this product
- labels is initially empty, we later fill it manually with `1` (relevant) or `0` (nonrelevant).

As before, the products are already ordered for each query:

	title	description	pid	query_id	labels
0	Men Slim Fit Checkered Casual Shirt	Men's checkered casual shirt featuring a colla...	SHTFZQEHAHE6VH3V	1	1
1	Men Slim Fit Checkered Casual Shirt	Men's casual checkered shirt featuring a colla...	SHTFH3MGMQTGPZRM	1	1
2	Men Regular Fit Checkered Casual Shirt	NaN	SHTEB2NY7ZTDGKQE	1	1
3	Men Slim Fit Checkered Casual Shirt	NaN	SHTFS6WHWGEDBZCW	1	1
4	Men Slim Fit Checkered Casual Shirt	NaN	SHTFS6WHPJRYDZKD	1	1

Figure 2. Example of the exported products and the relevance labels we assigned

We use only the top 20 retrieved products following the example of the provided validation_labels.csv. This gives us 100 products to label by hand. Ideally this should be larger, which is why companies that build real datasets hire people to manually label results one by one. We have not hired anyone to do this job.

b. Evaluation metrics for our test queries

All metrics are computed at $K = 10$ with binary relevance. They can be split into two groups.

Per-query metrics

Precision, recall and F1 only look at *which* products are in the top 10, not at *in which order*: swapping the 1st and the 10th product does not change them. **AP, RR and NDCG**

are rank-aware: they reward relevant products that appear higher in the list. Here is what each of them tells us about our 5 test queries.

Precision@10 is the fraction of the top 10 products that are relevant, so it tells us how "clean" the first page of results is. It is perfect for q1 and q2: the first 10 products are all relevant. In q3 it drops to 0.500: the products in positions 2-5 are "lace up" shoes and loafers, which rank high even though we asked for sports shoes, because the word *sports* appears in their description. Something similar happens in q4 (0.300), and in q5 (0.800) some irrelevant products are pushed up because the query words appear in their description.

Recall@10 is the fraction of all the relevant products that appear in the top 10, so it tells us about coverage. In q1 and q2 it is 0.500, but that is the best we can get: all 20 exported products are relevant, and the top 10 can contain at most $\frac{10}{20}$ of them. In q3 and q5 recall is also capped by K (at most 0.667 and 0.714). In q4 it is perfect because there are only 3 relevant products and all of them are in the top 10, but at positions 8, 9 and 10, which recall cannot see. Also note that recall only considers the 20 products we labelled. Any relevant product ranked below position 20 is unknown to us, so these values are optimistic, especially for q4.

AP@10 averages the precision at each position where a relevant product appears. It is like precision, but it also tells us *where* the relevant products are: it is 1 if they all come first (q1, q2), higher than P@10 when they are near the top, and lower when they are near the bottom. In q3 it goes from 0.500 to 0.563 because the first product is relevant, in q5 from 0.800 to 0.932 because the 2 irrelevant products are at positions 5 and 10, and in q4 from 0.300 to 0.216 because the 3 relevant products are at the end of the top 10.

F1@10 is the harmonic mean of precision and recall, so it summarizes both in one number and is low whenever one of them is low. It has two limits. First, it ignores the order. Second, since recall is measured at a fixed K, F1 is capped when there are more than 10 relevant products: q1 and q2 have a perfect ranking and still get 0.667, exactly the same F1 as q5, which has 2 irrelevant products in the top 10. In q4 the perfect recall is cancelled out by the low precision (F1 = 0.462), because the harmonic mean is closer to the lower value.

RR@10 is the inverse of the position of the first relevant product, so it tells us how fast a user finds *something* useful. It is 1 for q1, q2, q3, and q5 because the highest ranked

product is relevant. In q4 the first relevant product is in position 8, leading to an RR of $\frac{1}{8}$. It ignores everything after the first relevant result: q3 gets a perfect RR even though half of its top 10 is irrelevant.

NDCG@10 is also rank-aware, but it compares our ranking with the ideal one (all relevant products first) and discounts each product by the $\log_2$ of its position. Since it is normalized between 0 and 1, it is comparable across queries even if they have a different number of relevant products. It is perfect for q1 and q2 because our ranking is already the ideal one. In q5 (0.851) it is not perfect because an irrelevant product is placed in position 5, above 4 relevant ones. It is low in q3 (0.507) and q4 (0.425), where the relevant products are pushed down. NDCG is less harsh than AP in q4 (0.425 vs. 0.216) because the logarithmic discount decreases more slowly than $\frac{1}{position}$.

ID	query text	P@10	R@10	AP@10	F1@10	RR@10	NDCG@10
1	men's cotton checkered shirt	1.000	0.500	1.000	0.667	1.000	1.000
2	women casual shirt linen	1.000	0.500	1.000	0.667	1.000	1.000
3	black sports shoes	0.500	0.333	0.563	0.400	1.000	0.507
4	soft jacket for women	0.300	1.000	0.216	0.462	0.125	0.425
5	grey hoodie with pockets	0.800	0.571	0.932	0.667	1.000	0.851

Table 4. Per-query evaluation metrics for our queries.

Overall, the metrics agree that q1 and q2 are well solved, while q3 and q4 are the problematic ones. But they do not agree on how bad q3 is: MRR calls it perfect and NDCG calls it mediocre, which is why it is useful to look at several metrics together.

Aggregate metrics

MAP@10 is the mean of AP over all queries, so it gives a single number for the ranking quality of the system (0.742). Every query weighs the same: q1 and q2 are at the maximum, and q3 and q4 are the ones that pull it down.

MRR@10 is the mean of RR over all queries (0.825). Four of the five queries have a relevant first result, and q4 alone lowers it from 1.000 to 0.825. The reciprocal drops very fast ($1, \frac{1}{2}, \frac{1}{3}, \ldots$), so a single bad query has a large effect. MRR also says nothing about the rest of the ranking (q3 counts as perfect), so it is more suitable when one good result is enough than when users browse several products, where MAP and NDCG

are more informative.

MAP@10	MRR@10
0.742	0.825

Table 5. Aggregate evaluation metrics for our queries.

Precision-recall curves

In figure 3 we show the (interpolated) precision-recall curves for our fives tested queries. We can observe that q1 and q2 have always precision 1 because all their judged products are relevant. Q5 also performs well, although precision decreases as more relevant products are retrieved. Q3 remains at 0.75 after its initial drop because its later results improve precision, and it reaches fifteen relevant products in the end. Q4 is the one with the weakest curve, with only 0.20 precision.

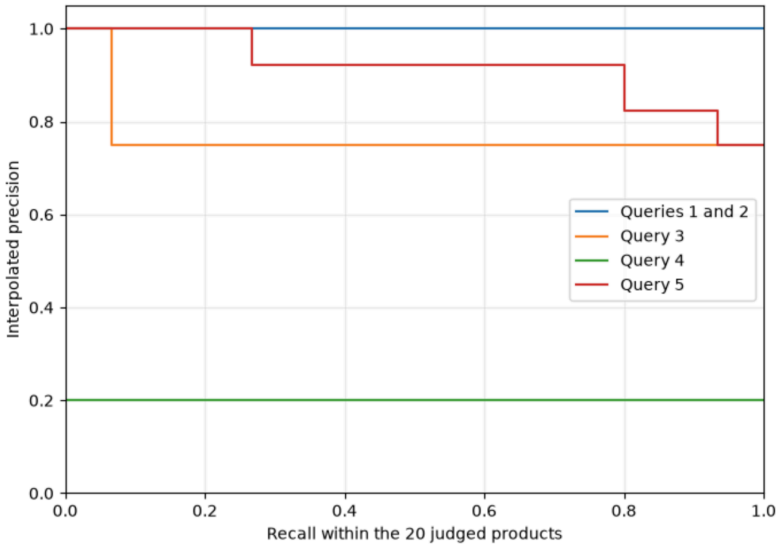


Figure 3. Interpolated precision–recall curves for our queries

c. Analysis of the current system

Limited evaluation scope: Note that this is just a small test with only 5 queries and a single annotator. We limited it to 20 retrieved products per query due to time constraints. Ideally this should be done with multiple annotators (measuring their agreement, e.g. with Cohen's κ), more queries, and all the retrieved products.

Equal weighting of fields: The metrics are based on equal weight for all fields. we did not tune them, but we have a potential solution to increase precision --> increase weight of titles. We have seen how some words appear in description that move irrelevant items to higher positions. In next labs we will take this approach.

Unequal performance. For some queries (q1, q2, q5), the system performs quite well. In contrast, the performance for q3 and q4 was pretty bad,

Similar products: near duplicate items in products dataset. Part of what went wrong in queries 3 and 4 is that many non-relevant products are retrieved but they are almost the same. e.g. for q3 = “soft jacket for women”, the top 4 result are copies of the same product with same title and description (“Women Ethnic Jacket and Kurta Set Cotton Jute Blend”), but with different PIDs. so essentially the system failed once, but since we have 4 copies of this product the results are pretty bad. We think this product may have matched the query because in product_details we have “Soft wash only” which obviously is not what we want. so this is another issue that could be solved by tuning field weights (giving less importance to product_details).

No query expansion: Right now the query must match exactly the words in the product. So for instance if we search for “sport shoes”, maybe we miss products that use the word “gym” or “running” instead of sport.

No user interface or numerical filtering: The system is currently implemented in a notebook: There is no ui, so a real user probably wouldn't be able to use it. In the database, we have numerical attributes such as price and rating that users could use to filter results, which would add up to the keyword matching of the queries.